v2.0.1

多Agent智能教育与个性化学习系统

从零到面试 · Java / Go / Python 三语言 · Mesh 与事件驱动

面试向多 Agent 教学参考实现：核心五类 Agent 经事件总线协作，结合 BKT 薄弱点诊断、苏格拉底式引导与 SM-2 复习排期；旁路含 Critic / Debate。资料含架构、配置、库表、部署、面试与知识点说明；工程侧提供 Python（功能最全）与 Java / Go 对照实现，便于简历与面试陈述。

Interview-oriented multi-agent education reference: five core agents on an event bus with BKT, Socratic tutoring, and SM-2 scheduling, plus Critic/Debate side paths. Docs cover architecture, config, schema, deployment, and interview prep; code includes a full Python stack and Java/Go counterparts.

文档版本： v2.0.1

发布时间： 2026-09-01

语言： 简体中文

> 「小番薯生活集」官方整理，认准本店，持续更新！

本产品包含的产品和工具信息，功能描述可能随时变化，请以各产品工具官方文档为准。

目录

Table of Contents

多Agent智能教育与个性化学习系统 1

架构设计详解 8

部署指南 15

配置文件说明 20

数据库及表结构设计 28

面试完全指南：多Agent智能教育系统 37

知识点体系 58

多Agent智能教育与个性化学习系统

> 面试向参考实现 | Java + Go + Python 三语言对照 | Mesh + 事件驱动

这个项目是什么?

这是一个可本地跑通的多 Agent 教育参考实现，用 Agent 协作做个性化教学演示：

- 按学生薄弱点调整教学节奏（BKT 掌握度）
- 用苏格拉底式提问引导思考，而不是直接给答案
- 用 SM-2 间隔重复动态排期复习

Python 为功能最全的参考后端（认证、RAG、题库、教师端、管理端等）；Java / Go 对照实现 Mesh、学习主链与旁路 Agent，路径与认证形态已大幅向 Python / 前端靠拢，但领域模型与 PG 全表域仍不完全一致，**不能当作可热切换的生产交付**。

为什么做这个项目?

1. 有一份可以在简历上写、面试时讲的多 Agent 项目经历
2. 理解 Mesh + 事件驱动的编排思想
3. 准备八股、STAR、简历写法材料
4. 用三种语言实现同一核心思路，适配不同岗位方向

系统架构

整体架构图

流量先进网关，再进 EventBus。Agent 只和总线、共享存储打交道，不在图里画 Agent 之间的斜线。Assessment / Tutor / Hint、Curriculum / Engagement 等协作都通过事件完成；Critic / Debate 为旁路（三端均有；Debate 需 `debate_mode`）。



Agent 职责

Agent	职责	核心算法/技术
Assessment	知识点掌握度评估	BKT、Beta 近似
Tutor	苏格拉底式教学	Prompt / 难度自适应
Curriculum	学习路径与复习排期	SM-2、知识图谱拓扑序
Hint	三级提示	概念 → 方法 → 步骤
Engagement	学习状态监测	挫败 / 疲劳 / 闲置等启发式
Critic	旁路评估 Tutor 回复	LLM-as-judge；低分阈值 `overall ⩽ 3`
Debate	多 persona 辩论	`debate_mode=true` 触发

Python 编排器启动时初始化上述 **7** 个 Agent。Go 另有 RAG / Question 等独立 Agent；SQLite 默认可落用户、掌握度、事件、业务 JSON、RAG 分块与 critiques。前端默认上游仍是 Python。

为什么用 Mesh + 事件驱动?

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

编排模式	特点	适用场景
Supervisor	中心化调度，易成瓶颈	简单串行任务流
Pipeline	线性流转，灵活性低	数据处理流水线
Mesh + 事件驱动	Agent 经总线解耦通信	需双向、异步交互的教学场景

选择 Mesh 的原因：

- Tutor 可发 `HINT_NEEDED`，Assessment 可通知 Curriculum，不必硬编码互相调用
- 新增 Agent 主要靠订阅事件，少改现有代码

核心算法

1. SM-2（Curriculum）

「快忘了的时候复习」一类间隔重复。实现见 `code/python/core/spaced_repetition.py`。

```
I(1)=1 天, I(2)=6 天, I(n)=I(n-1)×EF
EF' = EF - 0.8 + 0.28×q - 0.02×q² (q 为 0-5 质量分)
```

2. BKT（Assessment）

用概率估计掌握度。实现见 `code/python/core/learner_model.py`。

```
参数: P(L₀), P(T), P(G), P(S)
答对/答错后按贝叶斯更新 P(L)
```

3. 苏格拉底式教学（Tutor）

不直接给答案，用追问引导学生。实现见 `code/python/agents/tutor_agent.py`。

快速演示（Python + 前端）

演示账号（先执行 `python -m scripts.seed_demo_data`，密码均为 `demo123456`）：

角色	邮箱
学生	`student@demo.com`
教师	`teacher@demo.com`
管理员	`admin@demo.com`

联调注意：

1. 后端读 `code/python/.env`（可从 `code/.env.example` 复制）
2. 前端联调设 `VITE_USE_MOCK=false`（`.env.example` 默认是 `true`）
3. Python REST 仅 GET / POST（删除走 `POST .../delete`）
4. 步骤见 部署指南、配置文件说明

面试材料摘要

> 详见 面试完全指南：多Agent智能教育系统

简历示例（请按真实职责改写）

```
项目名称：多Agent智能教育与个性化学习系统
技术栈：FastAPI + 自定义 EventBus / Spring Boot 3 / Go / PostgreSQL / React / WebSocket
（LangGraph 仅用于 Critic/Debate 子图，非主编排）

- 设计 Mesh + 事件驱动：5 个核心教学 Agent + Critic/Debate 旁路，经 EventBus 协作
- Tutor 采用苏格拉底式 Prompt；Curriculum 用 SM-2；Assessment 用 BKT 并持久化掌握度
```

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

- Python 侧扩展认证、RAG、题库、教师端；前端对接 Python API
- WebSocket 推送学习事件

STAR（设计目标口径，非已测指标）

维度	内容
S	传统在线课内容同质化，困难时容易直接看答案或放弃
T	做可演示的多 Agent 个性化教学参考实现
A	Mesh + EventBus；BKT / SM-2 / 苏格拉底 Prompt；Python 扩展业务面
R	仓库未附带对照实验数据；可演示联调与算法行为，面试时勿编造保留率/准确率

高频问题

问题	要点
为何多 Agent?	职责隔离、并行、可独立扩展
为何 Mesh?	教学交互双向，非固定流水线
如何通信?	EventBus 发布-订阅
状态一致性?	Assessment 单写 mastery + 版本号等（以代码为准）

技术栈对照

维度	Python	Java	Go
框架	FastAPI	Spring Boot 3	标准库 `net/http`
Agent 编排	自定义 EventBus（asyncio）	Spring Event + 同步 `@EventListener`	goroutine + channel（Publish 等待订阅方）
LLM 子图	LangGraph（Critic/Debate）	服务内图/顺序调用	服务内顺序调用
实时	WebSocket	WebSocket	WebSocket + SSE
数据	SQLAlchemy + asyncpg	JPA + Flyway 全表域	SQLite（主数据 + biz/RAG/critiques）
与前端契约	主路径	主路径已对齐，领域细节仍有差	主路径已对齐，领域细节仍有差

参考开源 / 论文（星标勿写死，以 GitHub 实时为准）

项目 / 文献	参考点
CrewAI	Agent 协作模式
LangGraph	状态图（本项目仅旁路使用）
Solace Agent Mesh	事件驱动 Mesh
Spring Boot	Java 选型
IntelliCode 论文	教育多 Agent / 集中式学习者模型

学习路线

第一阶段：原理

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

1. 架构设计详解
2. 配置文件说明、数据库及表结构设计
3. 知识点体系 ↔ `build_sample_math_graph()`
4. 理解 SM-2 / BKT 直觉

第二阶段：代码

1. `code/python/` 为主
2. `core/event_bus.py` → `agents/base_agent.py` → 七个 `*_agent.py`
3. `api/main.py`、`api/orchestrator.py`
4. `cd code/python && python -m pytest tests/ -q`

第三阶段：改一改

1. 给知识图谱加节点
2. 改苏格拉底 Prompt
3. 调 SM-2 参数观察排期变化

第四阶段：面试

1. 面试完全指南
2. STAR 口述练习
3. 本地能按 部署指南 跑通演示

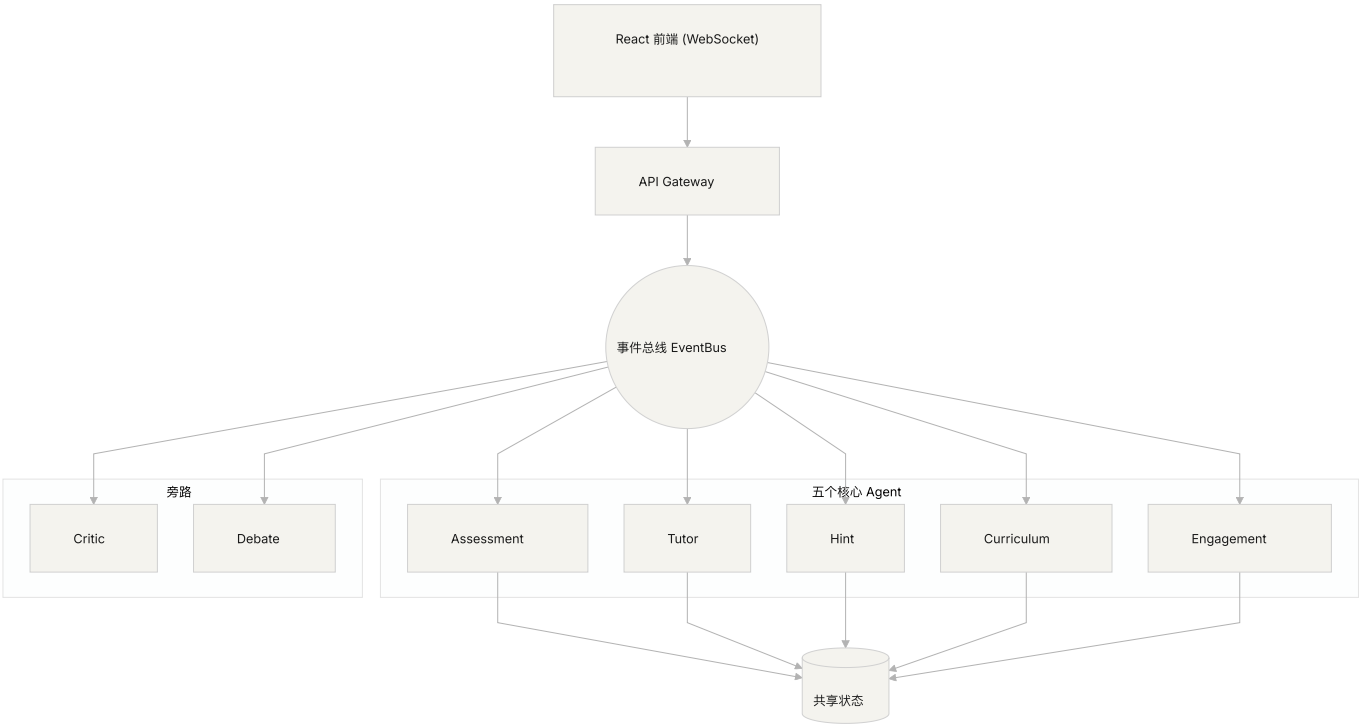
架构设计详解

本系统采用 **Mesh + 事件驱动**：核心教学 Agent 不互相硬编码直连，经 EventBus 发布/订阅协作。三端均含 Critic / Debate 旁路（Debate 由请求体 `debate_mode=true` 显式触发）。持久化：Python / Java 常用 PostgreSQL（+ Redis）；Go 默认 SQLite（`APP_SQLITE_PATH`，默认 `data/edu.db`）落用户、掌握度、事件、答题、审计、题库/班级/作业、RAG 分块与 Critic `critiques` 等。

与 概览 一致，读图时把「Mesh」理解成经总线的协作语义即可。

---

1. 整体架构



能力边界（避免误解「三语言完全对齐」）

- **编排思想一致**：网关 → 总线 → Agent → 存储；教学主链事件分发在三端均为同步等待（REST 返回前链已处理完）
- **业务面**：Python 最全（认证、RAG、题库、教师端、隐私、租户等）；Java / Go 已对齐主路径前缀（如 `/api/v1/knowledge-base`、`/tenants`、`/audit`）及学习主链契约（JWT 推断 `learner_id`、登录 Token 形态等）
- **前端契约**：以 Python + `code/frontend` 为准。Java / Go 仍有字段粒度、题库/班级领域模型与 PG 全表域差异，**不能只改 nginx 就当热切换生产后端**
- **HTTP**：Python 业务 API 约定为仅 GET / POST（见 配置文件说明 / `code/python/README.md`）

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

源码对照

概念	Python	Java	Go
事件类型	`core/event_bus.py`	`EventType` + `AgentEvent`	`eventbus`
Agent 基类	`agents/base_agent.py`	`@EventListener`	`agent/agents.go`
Critic / Debate	`critic_agent.py` / `debate_agent.py`	对应 Agent 类	对应 Agent
编排	`api/orchestrator.py`（7 Agent）	Spring IoC	`cmd/main.go`
知识图谱	`core/knowledge_graph.py`	`KnowledgeGraph`	`model/knowledge_graph.go`
BKT	`core/learner_model.py`	`LearnerModel`	`model/learner.go`
SM-2	`core/spaced_repetition.py`	`SpacedRepetition`	`model/learner.go`
实时	WebSocket	WebSocket	WebSocket + SSE
RAG 向量	JSON embedding + 内存 cosine（默认）；见 配置文件说明	Flyway 表域 + 服务层检索	Mock embedding + 内存向量；分块可落 SQLite `rag_chunks`
Critic 落库	表/ORM 已就绪（`critiques`）	`CritiqueRepository` 写库	SQLite `critiques`（评估后写入，重生成回写）
Critic 阈值	`overall ⩽ 3` 或缺省时触发重生成	同左（启发式）	同 Python：`needs_regeneration` 缺省时 `overall ⩽ 3`

事件类型速查（节选）

完整列表以各语言 EventType 为准；字符串值在语言间可能略有差异。

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

分组	枚举名（Python）	含义
学生侧	`STUDENT_SUBMISSION` / `STUDENT_QUESTION` / `STUDENT_MESSAGE`	提交、提问、消息
评估	`ASSESSMENT_COMPLETE` / `MASTERY_UPDATED` / `WEAKNESS_DETECTED`	评估、mastery、薄弱点
教学	`TEACHING_RESPONSE` / `HINT_NEEDED` / `DIFFICULTY_ADJUSTED`	回复、提示需求、难度
评判/辩论	`TEACHING_CRITICIZED` / `DEBATE_CONSENSUS`	Critic 结果、辩论共识
课程	`PATH_UPDATED` / `REVIEW_SCHEDULED` / `NEXT_TOPIC`	路径、排期、下一知识点
提示	`HINT_RESPONSE`	分级提示
互动	`ENGAGEMENT_ALERT` / `ENCOURAGEMENT` / `PACE_ADJUSTMENT`	预警、鼓励、节奏

2. 事件流设计

学生答题（示意）

```
学生答题 → STUDENT_SUBMISSION（`learner_id` 来自 JWT，不放 body）
  → Assessment → MASTERY_UPDATED → Curriculum（SM-2 / REVIEW_SCHEDULED）
  → ASSESSMENT_COMPLETE → Tutor / Engagement
  → Engagement 状态启发式 → 可能 ENGAGEMENT_ALERT / PACE_ADJUSTMENT /
ENCOURAGEMENT
  → Tutor 发布 TEACHING_RESPONSE → Critic 旁路打分 → TEACHING_CRITICIZED
  → （overall ≤ 3 或缺省规则命中）可再生成一条带 regenerated=true 的
TEACHING_RESPONSE
```

学生提问 / 辩论（示意）

```
STUDENT_QUESTION（可选 debate_mode=true）
  → Assessment / Tutor 正常处理
```

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

→ debate_mode=true 时 Debate 旁路 → DEBATE_CONSENSUS

学生卡住（示意）

连续答错 → Tutor 发 HINT_NEEDED → Hint → HINT_RESPONSE → Tutor 转发给学生

---

3. 为什么选择 Mesh

- **Supervisor**：中心调度易成瓶颈，难表达双向教学交互
- **Pipeline**：固定顺序，难支持「随时要提示 / 随时调难度」
- **Mesh**：经总线订阅即可扩展；新增 Agent 主要加订阅，少改旧代码

示意（伪代码）：

```python
class ParentNotifyAgent(BaseAgent):
    @property
    def subscribed_events(self):
        return [EventType.WEAKNESS_DETECTED, EventType.ENCOURAGEMENT]

    async def handle_event(self, event):
        ...
```

4. 三种语言的事件总线对比

维度	Python	Java	Go
总线	自定义 EventBus (asyncio)	Spring ApplicationEvent	channel + goroutine
订阅	`bus.subscribe`	`@EventListener`（教学 Agent 同步，无 `@Async`）	`bus.Subscribe`
主链等待	`asyncio.gather` 等待订阅方	同步监听 + 历史服务落库	`Publish` + WaitGroup 等齐
Agent 协程	asyncio Task	调用线程内执行	每 Agent 独立 goroutine 消费
实时	WebSocket	WebSocket	WebSocket + SSE
线程安全	asyncio 单线程协作	ConcurrentHashMap / RWLock 等	`sync.RWMutex`
外部依赖	见 `requirements.txt`	Spring 生态	`gorilla/websocket` + `golang.org/x/crypto` + modernc SQLite
可观测性	`/metrics` 等	Actuator	`/metrics`、`/metrics/prometheus`

5. 数据流

```
学生行为 → Assessment 更新 mastery → 持久化（Python/Java PG；Go SQLite）
  → submissions / events 落库（三端主链）
  → 事件通知 Curriculum / Tutor / Engagement
Curriculum → SM-2 排期、图谱拓扑推荐
Tutor → 苏格拉底回复；（可选）Critic 旁路打分
```

状态一致性（以 Python 设计意图为准）

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

- **单写者**：mastery 主要由 Assessment 写入
- **事件可追溯**：三端均可将总线事件落入 `events`（Python ORM；Java `EventHistoryService` 内存 last-N + 落库；Go EventBus persist → SQLite）
- **版本号**：`knowledge_states.version` 使用 CAS（`WHERE version=?` 成功后 +1）；冲突时有限次重试（Python / Java / Go 主链均已接）

知识图谱：`build_sample_math_graph()` 当前约 20 个示例节点（见 知识点体系）。更大题库是扩展方向，不是仓库现成规模。

RAG：Python 文档切块后 embedding 存 JSON，默认内存 cosine。`USE_PGVECTOR` 需列类型支持，当前 schema 默认不要开。Go 演示用 Mock embedding，向量在内存检索，分块可同步写入 SQLite。细节见 配置文件说明、数据库及表结构设计。

---

6. 运行时栈

Python

层次	技术
API	FastAPI；生产镜像可用 Gunicorn + Uvicorn workers
Agent	Orchestrator 初始化 7 个 Agent
数据	SQLAlchemy async + PostgreSQL；Redis 缓存/限流
实时	WebSocket

Java

层次	技术
API	Spring Boot；REST + WebSocket
数据	JPA；开发可用 H2，Compose 可挂 PostgreSQL
实时	WebSocket

Go

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

层次	技术
API	标准库 `net/http`；JWT 含 `learner_id`，学习主链从 token 推断学习者
数据	SQLite（`APP_SQLITE_PATH`）：users、knowledge_states、refresh_tokens、events、submissions、audit_logs、biz_entities（题库/班级/作业）、rag_chunks、critiques；Compose 不强制 postgres
实时	WebSocket + SSE
Docker	Distroless 静态二进制（体积小是镜像特性，非业务压测结论）

三种语言：**编排思想与主路径已大幅对齐，但领域模型粒度、PG 全表域与前端契约仍以 Python 为准**。联调与演示优先 Python + 前端。

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

部署指南

可运行工程在仓库 `code/`（与 VitePress 的 `docs/` 并列）。下文路径相对仓库根目录。

演示账号（密码均为 `demo123456`）：`student@demo.com` / `teacher@demo.com` / `admin@demo.com`。

方式一：本地开发（推荐）

Python API（功能最全）

```bash
cd code/python
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp ../.env.example .env   # settings 读本目录 .env
# 至少改 JWT_SECRET_KEY、DATABASE_URL；出题/RAG 再配 OPENAI_* / EMBEDDING_*
cd .. && docker compose up -d postgres redis && cd python
alembic upgrade head
python -m scripts.seed_demo_data
uvicorn api.main:app --reload --port 8000
# OpenAPI: http://localhost:8000/docs
```

前端联调

```bash
cd code/frontend
cp .env.example .env
# 必须: VITE_USE_MOCK=false
npm install && npm run dev
# 默认 http://localhost:3000，占用时 Vite 会顺延端口
```

Java / Go（对照实现）

```bash
cd code/java && mvn spring-boot:run      # :8080
cd code/golang && go run cmd/main.go     # :8081，Go 版本以 go.mod 为准
```

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

下列仅为学习主链路节选，完整端点以各语言 README / 源码为准。主路径前缀与认证形态已向 Python / 前端靠拢；领域字段与 PG 全表域仍可能有差。

方法	路径（示意）	说明
GET	`/api/v1/health`	健康检查
POST	`/api/v1/auth/login`	登录；返回 `access_token` / `refresh_token` / `token_type` / `expires_in_seconds` / `user.learner_id`
POST	`/api/v1/submit`	提交答题（`learner_id` 由 JWT 推断，body 不传）
POST	`/api/v1/question` / `/message`	提问（可带 `debate_mode`）/ 消息
GET/POST	`/api/v1/knowledge-base/...`	RAG（列表/检索响应形态对齐 Python：多为裸数组或文档对象）
GET	`/api/v1/progress/...`	进度
WS	`/ws/...`	WebSocket（Go 另有 SSE `/api/v1/stream/...`）

Go 默认 SQLite（`APP_SQLITE_PATH`，见上），不连 Compose 里的 Postgres。Java 可挂 PostgreSQL，启动会种子演示账号。路径与主链契约已向 Python 对齐，但仍建议以 Python 为前端默认上游。

可选：`GUNICORN_WORKERS`（默认 4）、`GUNICORN_TIMEOUT` 等（见 Dockerfile / 启动脚本）。

方式二：单镜像构建

Python

```bash
cd code
docker build -t multi-agent-education-python:2.0.0 -f python/Dockerfile
./python
docker run -d --name python-api -p 8000:8000 \
  -e DATABASE_URL=postgresql+asyncpg://postgres:secret@host:5432/edu_agent \
  -e REDIS_URL=redis://host:6379/0 \
```

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

```
  -e JWT_SECRET_KEY=change-me-at-least-32-chars \
  -e OPENAI_API_KEY=sk-xxx \
  -e EMBEDDING_API_KEY=sk-xxx \
  multi-agent-education-python:2.0.0
```

Go / Java

见各目录 Dockerfile。Go 镜像为 Distroless 静态二进制；Java 为分层 JAR。均需自行配置 JWT / DB（若使用）等。

方式三：Docker Compose 编排

文件：`code/docker-compose.yml`。

服务：`postgres`、`redis`、`migrate`（一次性 Alembic）、`python-api`、`go-api`、`java-api`、`frontend`。

默认 `docker compose up` 会拉起三个后端；前端 nginx 只反代 `python-api`。推荐先只起 Python 栈：

```bash
cd code
cp .env.example .env
# 必填：POSTGRES_PASSWORD、JWT_SECRET_KEY（无弱默认值）
docker compose up -d --build postgres redis migrate python-api frontend
# migrate 随依赖自动跑完；也可显式依赖链由 compose 处理
docker compose exec python-api python -m scripts.seed_demo_data
```

服务	地址
前端	http://localhost:3000
Python OpenAPI	http://localhost:8000/docs
Go / Java（若已启动）	`:8081` / `:8080`
PostgreSQL / Redis	`:5432` / `:6379`

「切换到 Go/Java」不是改一行 `proxy_pass` 就完事：题库/班级领域模型、PG 全表域与个别响应字段仍可能与前端期望有差。主路径（如 `/knowledge-base`、租户/审计前缀、JWT `learner_id`）已对齐，仅适合对照调试，不适合当作已验证的生产热切换。

Compose 环境变量注意

`code/.env` 供 Compose 插值；本机 uvicorn 读的是 `code/python/.env`，二者不是同一文件。

`python-api` 容器会注入 DB/Redis/JWT/CORS 以及 `OPENAI_*` / `EMBEDDING_*` 等（以 `docker-compose.yml` 为准）。若改了 compose 未列出的变量名，仍需自行补 `environment`。

变量	说明
`POSTGRES_PASSWORD`	必填
`JWT_SECRET_KEY`	必填
`REDIS_PASSWORD`	可选
`OPENAI_API_KEY`	可选；空则 LLM 能力降级
`CORS_ORIGINS`	默认常含 `http://localhost:3000`

Java/Go 也可读 `OPENAI_API_KEY`（映射为 `APP_LLM_API_KEY`）；无 Key 时用模板/启发式降级，不是「永远不用 LLM」。

联调检查清单

- [ ] `code/python/.env` 或 Compose `.env` 已设 JWT / DB
- [ ] 已 `alembic upgrade head`（或 Compose `migrate` 成功）
- [ ] 已 `seed_demo_data`，能用演示账号登录
- [ ] 前端 `VITE_USE_MOCK=false`
- [ ] 出题 / RAG 需要时 Embedding 维数与库内向量一致（种子脚本会校正演示数据）

环境要求

工具	版本
Python	3.11+
Java	21+（若跑 Java）
Go	以 `go.mod` 为准（若跑 Go）
Node.js	20+
Docker / Compose	24+ / v2

常见问题

1. 端口占用：改 compose `ports` 左侧，或停本机 Postgres/Redis。
2. 库连不上：容器内主机名是 `postgres`，本机调试用 `localhost`。
3. 前端全是假数据：检查是否仍为 `VITE_USE_MOCK=true`。
4. Compose 起了 Go/Java 但前端行为不变：前端默认只打 Python。
5. RAG 召回为空：常见原因是查询向量维数与库内 chunk 不一致，或 `min_score` 过高；见 配置文件说明 Embedding 节。
6. 只测 Agent：`cd code/python && python -m pytest tests/ -q`。

更多：架构设计详解 · 配置文件说明 · 数据库及表结构设计 · 面试完全指南 第六节。

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

配置文件说明

本文说明可运行工程 `code/` 中的环境变量与配置加载方式。权威源码：

文件	作用
`code/.env.example`	后端 / Compose 环境变量模板
`code/python/.env`	Python 运行时实际读取的文件（gitignore）
`code/python/config/settings.py`	Pydantic Settings，映射环境变量
`code/frontend/.env.example`	前端 Vite 环境变量模板
`code/docker-compose.yml`	容器内覆盖 `DATABASE_URL` / `REDIS_URL` 等

Java / Go 在未配置 LLM Key 时会走模板或启发式降级；配置了 `OPENAI_API_KEY`（Compose 映射为 `APP_LLM_API_KEY`）时可走 OpenAI 兼容接口。学习主链与 RAG/租户/审计等路径已向 Python 靠拢；完整教师端、隐私作业流与前端契约仍以 **Python** 为准，不要假定三端可热切换。Java 版使用 Spring Boot 配置（`application.yml` + 环境变量覆盖），Go 版使用环境变量（含 `APP_SQLITE_PATH`）。

---

1. 如何加载

Python

Settings 固定读取 `code/python/.env`（相对 `config/settings.py` 定位，不依赖当前工作目录）：

```bash
cd code/python
cp ../.env.example .env
# 编辑 JWT_SECRET_KEY、DATABASE_URL、OPENAI_* / EMBEDDING_* 等
```

也可把同名变量直接注入进程环境（Docker Compose、`export`），优先级高于 `.env` 文件。

前端

```bash
cd code/frontend
cp .env.example .env
```

仅 `VITE_` 前缀变量会打进前端产物。改完后需重启 `npm run dev`。

Docker Compose

在 `code/` 下：

```bash
cp .env.example .env
docker compose up --build -d
```

Compose 读取 `code/.env` 做变量插值；本机 uvicorn 读取 `code/python/.env`——两套文件，不要混用期望。

`python-api` 服务当前透传：`DATABASE_URL`、`REDIS_URL`、`OPENAI_*`（含 model/base_url/vision）、`MINIMAX_API_KEY`、`EMBEDDING_*`、`JWT_SECRET_KEY`、`CORS_ORIGINS`、`ENV`、`LOG_LEVEL`。根目录 `.env` 里写了但 compose 未列入的变量，容器仍读不到。库内主机名用 `postgres` / `redis`，本机调试用 `localhost`。

---

2. Python / 后端变量一览

下列名称与 Settings 字段对应（环境变量一般为大写 + 下划线）。

2.1 LLM（对话 / 出题 / Vision）

变量	默认倾向	说明
`OPENAI_API_KEY`	空	OpenAI 兼容接口密钥；空且非 mock 时部分能力降级
`OPENAI_MODEL`	`gpt-4o`	聊天模型
`OPENAI_BASE_URL`	`https://api.openai.com/v1`	可改为 DashScope compatible-mode 等
`OPENAI_VISION_MODEL`	空（按 base_url 推断）	拍题 / 识图；DashScope 常见 `qwen-vl-plus`
`MINIMAX_API_KEY` / `MINIMAX_MODEL`	可选	国产合规备选
`LLM_TIMEOUT_SECONDS`	`30`	单次调用超时
`LLM_MAX_RETRIES`	`3`	重试次数
`LLM_PROVIDER`	空	设为 `mock` 强制 Mock LLM（测试常用）

2.2 Embedding（RAG + 题目去重）

与聊天模型可分离；字段留空时回退到 `OPENAI_API_KEY` / `OPENAI_BASE_URL`，模型按 base_url 推断。

变量	说明
`EMBEDDING_API_KEY`	Embedding 专用密钥
`EMBEDDING_BASE_URL`	如 DashScope `.../compatible-mode/v1`
`EMBEDDING_MODEL`	如 `text-embedding-v3` / `text-embedding-3-large`
`EMBEDDING_DIMENSIONS`	`0` 表示按模型默认；显式值会随请求传给支持维度的接口

注意：部分供应商可能忽略请求中的 `dimensions`，以 API 实际返回维数为准。种子脚本 `seed_demo_data` 会按实际维数校正演示文档向量，避免与查询向量不一致导致召回全空。

可选：`USE_PGVECTOR=1` 时尝试 PgVectorStore。当前 Alembic schema 中 `embedding` 为 JSON 数组，没有现成 `vector` 列；默认请保持关闭，使用内存 cosine。误开可能导致 SQL 失败或召回异常。

2.3 数据库与 Redis

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

变量	示例 / 默认	说明
`DATABASE_URL`	`postgresql+asyncpg://postgres:postgres@localhost:5432/edu_agent`	必须使用 asyncpg 驱动前缀
`DB_POOL_SIZE`	`20`	连接池大小
`DB_MAX_OVERFLOW`	`10`	溢出连接
`DB_POOL_TIMEOUT`	`30`	取连接超时（秒）
`DB_POOL_RECYCLE`	`1800`	连接回收（秒）
`DB_ECHO`	`false`	是否打印 SQL
`REDIS_URL`	`redis://localhost:6379/0`	缓存 / 限流
`REDIS_PASSWORD`	空	Redis 密码（可选，生产建议设置）

测试环境常见覆盖：`DATABASE_URL=sqlite+aiosqlite:///:memory:`、`ENV=test`。

2.4 Java / Go 后端变量

Java（Spring Boot）通过 `application.yml` + 环境变量覆盖（`${VAR:default}` 语法）：

变量	示例 / 默认	说明
`DB_URL`	`jdbc:postgresql://localhost:5432/edu_agent`	JDBC 连接串
`DB_USERNAME`	`postgres`	数据库用户名
`DB_PASSWORD`	`postgres`	数据库密码
`SPRING_PROFILES_ACTIVE`	`dev`	激活的 profile（`prod` 启用 Flyway validate）
`JWT_SECRET_KEY`	`CHANGE_ME_IN_PRODUCTION`	JWT 签名密钥（生产必须更换）
`RATE_LIMIT_ENABLED`	`true`	限流总开关

Go 通过环境变量读取：

变量	示例 / 默认	说明
`APP_JWT_SECRET`	`change-me`	JWT 签名密钥（生产必须更换）；access token 含 `learner_id`
`APP_SQLITE_PATH`	`data/edu.db`	SQLite 路径；设为 `:memory:` 则不落盘文件
`APP_LLM_API_KEY`	空	LLM API Key（空则模板 / 启发式降级）
`APP_LLM_BASE_URL`	`https://api.openai.com/v1`	LLM 端点
`APP_LLM_MODEL`	`gpt-4o-mini`	模型名
`APP_LLM_TIMEOUT`	`30`	单次调用超时（秒）
`APP_LLM_MAX_RETRIES`	`3`	重试次数
`APP_LLM_CB_THRESHOLD`	`5`	熔断器阈值（连续失败次数）
`APP_LLM_CB_DURATION`	`60`	熔断冷却期（秒）
`APP_LLM_PROVIDER`	`auto`	提供者选择（`auto` / `openai` / `mock`）

2.5 认证与 CORS

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

变量	说明
`JWT_SECRET_KEY`	生产必须更换；审计 HMAC 在未单独配置时也可能回退使用
`JWT_ALGORITHM`	默认 `HS256`
`JWT_ACCESS_TOKEN_EXPIRE_MINUTES`	默认 `30`
`JWT_REFRESH_TOKEN_EXPIRE_DAYS`	默认 `7`
`CORS_ORIGINS`	`*` 或逗号分隔源列表

相关（可不进 `.env.example` 但仍可读环境）：

- `AUDIT_HMAC_SECRET`：审计链专用密钥（建议与 JWT 分离）
- `PII_ENCRYPTION_KEY`：PII 字段加密（未配时开发环境有降级逻辑，生产应显式配置）

2.6 限流

变量	默认	说明
`RATE_LIMIT_ENABLED`	`true`	总开关
`RATE_LIMIT_REDIS_URL`	空	空则用内存计数器
`RATE_LIMIT_GLOBAL_PER_MIN`	`100`	按 IP 全局限流
`RATE_LIMIT_LOGIN_PER_MIN`	`5`	登录防爆破
`RATE_LIMIT_REGISTER_PER_MIN`	`3`	注册
`RATE_LIMIT_REFRESH_PER_MIN`	`10`	刷新 token

2.7 运行时与可观测性

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

变量	说明
`ENV`	`dev` / `staging` / `prod`；`test` / `ci` 走 Mock
`DEBUG`	`true` 时等同开发友好日志
`LOG_LEVEL`	如 `INFO`
`API_PORT`	默认 `8000`（也可用 `uvicorn --port`）
`OTEL_EXPORTER_ENDPOINT`	OTLP gRPC 地址；空则关闭 tracing
`ENABLE_TRACING`	是否启用 OpenTelemetry

Compose / 模板中的 `PYTHON_API_PORT`、`FRONTEND_PORT` 等主要用于编排说明，应用内以 `API_PORT` 与 Vite / Nginx 端口为准。

3. 前端变量

变量	说明
`VITE_USE_MOCK`	`true`：MSW 纯前端演示，且不把 `/api` 代理到真实后端
`VITE_API_BASE`	默认 `/api/v1`；联调时经 Vite 代理到 Python `:8000`
`VITE_WS_BASE`	可选；如 `ws://localhost:8000`，直连后端 WebSocket

联调推荐：

```env
VITE_USE_MOCK=false
VITE_API_BASE=/api/v1
```

仓库示例默认常为 `VITE_USE_MOCK=true`（只开 UI 演示）。

4. 推荐最小配置

本地联调（真实 LLM + RAG）：

1. 前端 `.env`：`VITE_USE_MOCK=false`
2. 启动 postgres / redis

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

3. `code/python/.env` 至少：`JWT_SECRET_KEY`、`DATABASE_URL`；出题/对话配 `OPENAI_*` 或 MiniMax；RAG/去重建议 `EMBEDDING_*`
4. `alembic upgrade head` → `python -m scripts.seed_demo_data`
5. 演示账号（密码 `demo123456`）：`student@demo.com` / `teacher@demo.com` / `admin@demo.com`

仅看 UI：保持 `VITE_USE_MOCK=true`，无需数据库与密钥。

---

5. 与部署文档的关系

端口、Compose 服务名、常见排错见 部署指南。表结构与迁移见 数据库及表结构设计。源码侧 README：`code/README.md`、`code/python/README.md`、`code/frontend/README.md`。

数据库及表结构设计

本文描述 Python 参考实现（`code/python`）的持久化设计。Java 经 Flyway 覆盖全表域，主链路写回 `knowledge_states`（CAS）、`submissions`、`events`，Critic 写 `critiques`。Go 用 SQLite（`APP_SQLITE_PATH`，默认 `data/edu.db`）落：users、knowledge_states、refresh_tokens、events、submissions、audit_logs、`biz_entities`（题库/班级/作业 JSON）、`rag_chunks`、`critiques`。三端演示账号约定一致（`*@demo.com` / `demo123456`）。

项	说明
引擎	PostgreSQL 16（生产 / 演示）；测试可用 SQLite in-memory；Go 运行时默认独立 SQLite 文件
ORM	SQLAlchemy 2.0 async + asyncpg
迁移	Alembic：`migrations/versions/0001` – `0010`
模型源码	`db/models.py`
多租户	共享库 + 业务表 `tenant_id` 行级隔离；知识图谱节点全局共享

1. 设计原则

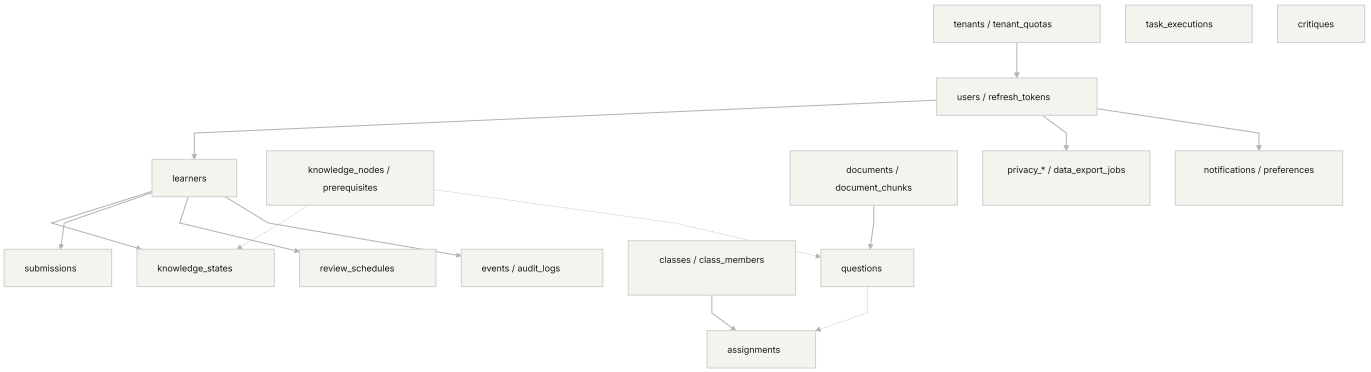
1. **学习者状态可恢复**：BKT 掌握度、SM-2 排期、答题记录落库，进程重启不丢。
2. **事件可追溯**：总线事件写入 `events`；敏感操作走 HMAC 链 `audit_logs`。
3. **向量跨方言**：`document_chunks.embedding` / `questions.content_embedding` 存 JSON 数组，默认内存 cosine 检索；可选日后迁 pgvector。
4. **租户隔离**：除全局知识图谱外，业务行带 `tenant_id`（默认 `1`）。
5. **状态机显式化**：题目、作业、删除申请、导出任务等用字符串状态字段表达生命周期。

2. 逻辑关系总览

虚线表示逻辑关联（如作业里的 `question_ids` JSON、掌握度里的 `knowledge_id` 字符串），不一定是数据库外键。

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！



3. 迁移演进

版本	内容
`0001`	学习者、知识图谱、掌握度、提交、复习、事件、审计等基础表
`0002`	`users` / `refresh_tokens`
`0003`	RAG：`documents` / `document_chunks`
`0004`	`questions`
`0005`	`classes` / `class_members` / `assignments`
`0006`	隐私同意、删除申请、数据导出
`0007`	`task_executions`
`0008`	`notifications` / `notification_preferences`
`0009`	`tenants` / `tenant_quotas`，业务表补 `tenant_id`
`0010`	`critiques`

本地：

```bash
cd code/python
alembic upgrade head
python -m scripts.seed_demo_data    # 演示账号 + RAG 向量等
```

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

演示账号（密码均为 `demo123456`）：`student@demo.com` / `teacher@demo.com` / `admin@demo.com`。

---

4. 表分组说明

4.1 认证与学习者

tenants / tenant_quotas

表	要点
`tenants`	`name`、`slug`（唯一）、`plan`（free/pro/enterprise）、`settings` JSON
`tenant_quotas`	每租户一行：`max_users`、月度 LLM token 上限与已用量、`period_start`

users / refresh_tokens / learners

表	要点
`users`	`email` 唯一、`password_hash`、`role`（枚举含 student/teacher/admin/parent/auditor；前端主路径用前三）、`tenant_id`、可选 `learner_id`、登录失败计数
`refresh_tokens`	`token_hash`、`expires_at`、`revoked`
`learners`	教学侧主体：`public_id`（对外 ID）、`name`、`profile` JSON、`tenant_id`

学生账号通常 `users.learner_id` → `learners.id`；教师可有演示用 learner，也可无。

4.2 知识图谱与学习状态

knowledge_nodes / knowledge_prerequisites

- 节点：`public_id`、`name`、`subject`、`difficulty`、`tags`、`version`
- 前置：复合主键 `(node_id, prerequisite_id)`，构成 DAG
- 知识图谱不按租户拆分，各租户共用同一套节点定义

knowledge_states

每学习者 × 知识点一行（业务上由 `learner_id` + `knowledge_id` 定位）：

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

字段	含义
`mastery`	掌握度
`alpha` / `beta`	BKT / Beta 相关参数
`attempts` / `correct_count` / `streak`	统计
`version`	乐观锁，upsert 自增
`last_attempt`	最近作答时间

submissions

答题流水：`learner_id`、`knowledge_id`、`is_correct`、`time_spent_seconds`。

review_schedules

SM-2 排期：`easiness_factor`、`interval_days`、`repetition`、`next_review`、`last_review`。

4.3 事件与审计

events

总线持久化：`public_id`、`type`、`source`、`learner_public_id`、`data` JSON、`correlation_id`、`timestamp`。

audit_logs

链式哈希：`actor`、`action`、`target`、`ip`、`user_agent`、`metadata`、`prev_hash`、`curr_hash`。演示种子经 AuditService 写入以保证 HMAC 一致。

4.4 RAG 知识库

documents

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

字段	说明
`title` / `source_type` / `source_url`	标题与来源标签；入库 API 只收粘贴文本，无 PDF/DOCX 二进制解析（`pdf` / `docx` 仅为字段预留）；`source_url` 多为备注
`content_hash`	SHA-256，全文去重
`status`	`pending` / `indexing` / `indexed` / `failed`
`total_chunks` / `error_message`	分块数与失败原因
`metadata`	扩展（如 `seed: demo`）

document_chunks

字段	说明
`document_id` + `chunk_index`	唯一约束
`content`	分块正文
`embedding`	float[] 的 JSON
`token_count`	估算 token

级联：删文档删分块。

4.5 题库

questions

字段	说明
`knowledge_id`	关联知识点（字符串 ID）
`question_type`	`multiple_choice` / `fill_blank` / `true_false` / `open_ended`
`difficulty`	1–5
`content`	JSON：题干、选项等；选择题选项形如 `{ "key":"A","text":"..." }`
`answer` / `analysis`	标准答案与解析
`status`	`draft` → `review` → `published` → `archived`（另有 `rejected`）
`content_embedding`	可选，语义去重
`created_by`	创建者 `user_id`

4.6 班级与作业

classes / class_members

- 班级：`teacher_id`、`invite_code`（唯一）、`is_active`
- 成员：`(class_id, learner_id)` 唯一；学生通过邀请码加入

assignments

字段	说明
`class_id` / `teacher_id`	归属
`question_ids`	题目 ID 列表（JSON，非关联表）
`status`	`draft` → `published` → `closed`（及归档等扩展以代码为准）
`due_at`	截止时间

4.7 隐私合规

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

表	说明
`privacy_consents`	追加式同意记录（`consent_type` + `granted` + 版本）；当前状态取最新一行
`privacy_deletion_requests`	删除申请：`pending` → `scheduled` → `executing` → `completed` / `cancelled`；含宽限期
`data_export_jobs`	导出任务：`pending` → `running` → `completed` / `failed`，含 `download_url`

4.8 任务、通知、Critic

task_executions

定时 / 手动任务历史：`task_name`、`status`、`duration_ms`、`triggered_by`（scheduler/manual/webhook）。

notifications / notification_preferences

- 通知：`type`、`channel`（ws/email/push）、`status`、`priority`、`title` / `body` / `payload`
- 偏好：每用户一行，渠道开关、勿扰时段（UTC 小时）、`min_priority`

critiques

Critic 对 Tutor `TEACHING_RESPONSE` 的评分记录：`overall`（1–5）、`scores` JSON（pedagogical / appropriateness / clarity / engagement）、`critique` 文本、`needs_regeneration`、`original_response` / `regenerated_response`、`critic_model` / `critic_tokens`、`teaching_event_id`。

阈值约定（与 Python `CriticLLMService` 一致）：JSON 未给出 `needs_regeneration` 时，按 `overall ⩽ 3` 判定需重生成；若字段显式给出则以其为准。Java / Go CriticAgent 评估后写入本表（或 Go SQLite 同名表）；Python 侧表与 ORM 已就绪，运行时写入以当前源码为准。见 架构设计详解。

---

5. 核心字段速查（高频表）

knowledge_states

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

```
id, tenant_id, learner_id, knowledge_id,
mastery, alpha, beta, attempts, correct_count, streak,
last_attempt, version, updated_at
```

questions.content（选择题示例）

```json
{
  "stem": "分解因式 x^3 - 4x 的结果是（ ）",
  "options": [
    { "key": "A", "text": "x(x^2 - 4)" },
    { "key": "B", "text": "x(x - 2)^2" },
    { "key": "C", "text": "x(x + 2)(x - 2)" },
    { "key": "D", "text": "(x^2 + 2x)(x - 2)" }
  ],
  "answer_key": "C"
}
```

documents.status

`pending`（未索引）→ `indexing` → `indexed` | `failed`。

---

6. 索引与约束（实现要点）

- 邮箱、邀请码、文档 `content_hash`、租户 `slug`：唯一索引
- 高频过滤：`tenant_id`、`status`、`knowledge_id`、`learner_public_id`、事件 `type`
- 分块：`UNIQUE(document_id, chunk_index)`
- 班级成员：`UNIQUE(class_id, learner_id)`
- 外键删除：班级 / 文档等对子表多使用 `ON DELETE CASCADE`

具体索引以各 migration 与 `__table_args__` 为准。

7. 与运行时的关系

运行时组件	读写表
AuthService	`users`、`refresh_tokens`、`learners`
LearnerModelService / Assessment	`knowledge_states`、`submissions`
Curriculum / SM-2	`review_schedules`
EventBus persist	`events`
RAGService	`documents`、`document_chunks`
QuestionService	`questions`
Class / Assignment 服务	`classes`、`class_members`、`assignments`
Privacy / Audit / Task / Notification	对应 `privacy_*`、`audit_logs`、`task_executions`、`notifications*`
CriticAgent	`critiques`（Java / Go 运行时写入；Python 有迁移与 ORM）
（Go 扩展）	SQLite `biz_entities`、`rag_chunks`：题库/班级/作业与 RAG 分块落盘，非 PG 全量对照表

配置项（连接串、池大小等）见 配置文件说明。部署与种子账号见 部署指南。

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

面试完全指南：多Agent智能教育系统

> 涵盖：八股文50题 + STAR法回答模板 + 简历写法 + 项目问答 + 面试技巧

目录

- 一、简历怎么写
- 二、STAR面试法模板
- 三、八股文50题精选
- 四、面试常见项目问题及回答
- 五、面试注意事项
- 六、仓库与本地演示

一、简历怎么写

推荐写法（Python方向）

```
项目名称：多Agent智能教育与个性化学习系统
技术栈：FastAPI + 自定义 EventBus / PostgreSQL / Redis / React / WebSocket
（LangGraph 仅用于 Critic/Debate 子图）
时间：2026.02 - 2026.04

- 设计 Mesh+事件驱动（5 核心教学 Agent + Critic/Debate 旁路），Agent间通过EventBus双向异步通信，支持实时状态同步
- Tutor Agent采用苏格拉底式教学Prompt策略，优先暗示+引导，非直接给答案
- Curriculum Agent基于SM-2间隔重复算法动态调整学习路径，优化复习时机
- Assessment Agent使用贝叶斯知识追踪(BKT)模型，持久化学习者模型，跟踪知识点掌握度
- 采用WebSocket实现师生实时交互
```

推荐写法（Java方向）

项目名称：多Agent智能教育与个性化学习系统
技术栈：Spring Boot 3 / Spring Event / PostgreSQL / React / WebSocket
时间：2026.02 - 2026.04

- 设计 Mesh+事件驱动（5 核心 + Critic/Debate 旁路），基于 Spring ApplicationEvent 同步分发（教学 Agent 用 `@EventListener`，与 Python gather 语义对齐），correlation_id 链路追踪
- Tutor Agent 采用 4 级苏格拉底式教学策略（beginner/developing/proficient/mastered），优先引导而非直接给答案
- Engagement Agent 实现 6 状态学习监测（Focused/Struggling/Frustrated/Bored/Fatigued/Idle），疲劳检测(45min)、闲置检测(300s)
- Curriculum Agent 集成 SM-2 间隔重复 + 知识图谱拓扑排序，自动推送 REVIEW_SCHEDULED 复习排期
- Critic 旁路：LLM-as-judge，`overall ≤ 3` 触发自反思重生成，评估写入 `critiques`
- JPA + Flyway 落 mastery（CAS）/ submissions / events；WebSocket 实时通信；EventHistoryService 提供 last-N + 落库

推荐写法（Go方向）

项目名称：多Agent智能教育与个性化学习系统
技术栈：Go（以 go.mod 为准） / goroutine+channel / net/http / SQLite / React / WebSocket
时间：2026.02 - 2026.04

- 设计 CSP/goroutine Mesh（5 核心 + Critic/Debate 旁路），每个 Agent 独立 goroutine，channel 总线通信，Publish 等待订阅方完成（对齐 Python 主链语义）
- JWT 含 learner_id；/submit /question /message 从 token 推断学习者，响应形态向 Python / 前端对齐
- SQLite（APP_SQLITE_PATH）落 users、mastery、events、题库/班级、RAG 分块与 critiques；演示账号种子与 Python 一致
- Critic 阈值与 Python 一致（overall ≤ 3）；Debate 由 debate_mode 显式触发
- WebSocket（gorilla/websocket）+ SSE；可观测性：/metrics(JSON) + /metrics/prometheus
- sync.RWMutex 保证并发安全

简历注意事项

1. **结果表述要诚实**：可写「可演示联调 / 算法行为可观察」；不要编造未测过的保留率、准确率、并发 QPS
2. **突出架构决策**：为什么选 Mesh 不选 Supervisor？为什么用 EventBus？
3. **体现深度**：写清 BKT、SM-2 等算法名，并说明谁写 mastery
4. **条目精简**：项目经历控制在 5–6 条 bullet

二、STAR面试法模板

模板1：项目概述（必问）

S (Situation 情境)： 在线教育平台发现：传统教学系统"千人一面"，所有学生看到同样的内容，学生遇到困难时只能看答案或放弃。

T (Task 任务)： 我负责设计一个多Agent智能教育系统，核心目标：

1. 实时评估每个学生对每个知识点的掌握程度
2. 用苏格拉底式提问引导学生思考，而非直接给答案
3. 动态调整学习路径和复习计划
4. 监测学习状态，防止学生因挫败而放弃

A (Action 行动)：

1. **架构选型**：调研了CrewAI、LangGraph、Solace Agent Mesh等常见开源 Agent/编排框架，最终采用 Mesh+事件驱动总线（核心 5 Agent + Critic/Debate）
2. **为什么Mesh而非Supervisor**：教育场景需要Agent双向实时交互（如Tutor随时请求Hint），Supervisor的中心化调度是瓶颈
3. **核心算法**：Assessment Agent用BKT贝叶斯知识追踪，Curriculum Agent用SM-2间隔重复+知识图谱拓扑排序
4. **教学策略**：Tutor Agent设计了4级苏格拉底式Prompt模板（beginner/developing/proficient/mastered），Hint Agent实现三级提示策略
5. **状态监测**：Engagement Agent实现6状态学习监测（Focused/Struggling/Frustrated/Bored/Fatigued/Idle），疲劳检测、闲置检测、PACE_ADJUSTMENT节奏调整

6. **工程实现**：Python/Java/Go三语言实现，Java版WebSocket实时通信，Go版WebSocket+SSE双通道

R (Result 结果)：

- 可演示引导式教学链路（暗示 / 引导优先于直接给答案）
- 可观察 SM-2 排期与 BKT mastery 更新行为
- 无仓库内对照实验支撑的「思考时间提升 / 保留率提升 / 并发容量」数字；面试勿编造

模板2：技术难点（常问）

S：多Agent之间的状态一致性是一个关键挑战。多 Agent 并发读写学习者状态，如何保证数据一致?

T：设计一个既高效又一致的状态管理方案。

A：

1. 采用"单写者策略"：每个知识点的mastery只由Assessment Agent更新，其他Agent只读
2. 乐观锁+版本号：每次更新带version字段，防止并发写冲突
3. 事件溯源：所有状态变更通过事件记录，可以回溯和审计
4. 参考了IntelliCode（EACL 2026论文）的集中式学习者模型设计

R：设计上事件可追溯，便于排查；是否「Debug 效率提升」取决于团队流程，非本仓库测量结论

模板3：关于协作/选型决策（加分项）

S：团队对Agent编排模式有分歧——有人倾向Supervisor集中调度，有人倾向完全去中心化。

T：做技术选型决策，说服团队采用最适合教育场景的方案。

A：

1. 我对比了三种模式的优缺点（Supervisor/Pipeline/Mesh），画出了每种模式下的事件流图
2. 用具体场景说明：当学生答错 → Assessment更新mastery → Tutor需要实时知道 → 同时Hint要准备 → Engagement要监测，这种多方向同时交互，Supervisor的串行调度效率太低
3. 最终选择Mesh+事件驱动：Agent之间通过EventBus双向通信，每个Agent只订阅自己关心的事件

R：团队达成共识，架构可扩展性验证：后续新增"家长通知Agent"只需3行代码（订阅事件即可），无需修改任何现有Agent

---

三、八股文50题精选

A. 多Agent系统基础 (10题)

1. 什么是多Agent系统？和单Agent有什么区别?

多Agent系统(MAS)由多个具有独立能力的Agent协作完成任务。与单Agent的区别：

- **上下文隔离**：每个Agent有独立的System Prompt和上下文窗口，避免指令冲突
- **专业分工**：每个Agent专注自己的领域（评估/教学/提示），比一个"全能Agent"更精准
- **并行执行**：多个Agent可以同时处理不同方面，提高吞吐量
- **独立扩展**：可以单独升级或替换某个Agent，无需改动整体系统
- **容错性**：一个Agent出错不影响其他Agent继续工作

2. 常见的多Agent编排模式有哪些？各适用什么场景?

模式	特点	适用场景	代表框架
Supervisor (监督者)	中心调度，简单可控	串行任务流、简单分工	CrewAI Crews
Pipeline (管道)	线性流转，上游输出=下游输入	数据处理、ETL	LangChain LCEL
Mesh (网格)	双向通信，去中心化	实时交互、复杂协作	Solace Agent Mesh
Hierarchical (层级)	多层管理，逐级委派	大型组织、多团队	AutoGen
Swarm (群体)	自主协调，无固定结构	探索性任务	OpenAI Swarm

3. 为什么本项目选Mesh+事件驱动，而不是Supervisor?

教育场景的特殊性：

- Agent间需要**双向实时交互**：Tutor随时请求Hint，Assessment通知Curriculum
- **非线性流程**：学生的学习路径不是固定的A→B→C，而是根据表现动态调整
- **松耦合需求**：新增Agent（如家长通知Agent）不应该修改现有代码
- Supervisor的中心调度在这种场景下是**瓶颈**和**单点故障**

4. 事件驱动架构的优缺点?

优点：

- 松耦合：发布者不关心谁订阅，订阅者不关心谁发布
- 可扩展：新Agent只需订阅事件，不修改现有代码（开闭原则）
- 异步非阻塞：发布事件后不等待处理结果，提高吞吐量
- 事件溯源：所有事件有记录，方便Debug和审计

缺点：

- 调试困难：事件流是异步的，难以追踪完整调用链
- 最终一致性：不是强一致，需要处理状态同步问题
- 事件风暴：设计不当可能导致事件无限循环

5. 什么是发布-订阅模式？和观察者模式的区别?

发布-订阅(Pub/Sub)：发布者和订阅者通过中间件（EventBus/消息队列）解耦。发布者不知道订阅者是谁，订阅者不知道发布者是谁。

观察者模式：Subject直接通知Observer，是一对多的直接关系。

区别：Pub/Sub有中间件解耦，观察者模式是直接引用。本项目用的是Pub/Sub。

6. Agent的Memory如何设计?

两种记忆：

- **短期记忆**：当前对话上下文（存储在Agent内部或Redis），包含最近的交互历史
- **长期记忆**：学习者模型（存储在PostgreSQL），包含知识点mastery、学习偏好、历史数据

本项目中，LearnerModel就是长期记忆的核心，每次交互都更新并持久化。

7. Agent之间的通信方式有哪些?

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

- **消息传递**：直接发送消息（如gRPC/HTTP调用），耦合度高
- **共享状态**：通过共享数据库/Redis读写，需要处理并发
- **事件总线**：发布-订阅模式，本项目使用的方式
- **黑板系统**：所有Agent读写共享黑板，适合协商决策

8. 如何防止Agent间的事件循环?

- 事件携带`correlation_id`追踪事件链
- 设置最大事件链长度（如10），超过则终止
- 每个Agent记录已处理的事件ID，避免重复处理
- 使用状态机控制Agent的处理状态

9. 什么是LangGraph？它的核心概念是什么?

LangGraph是LangChain团队开发的Agent编排框架，核心概念：

- **StateGraph**：有向图，节点是处理函数，边是状态转移
- **State**：共享状态对象，所有节点可以读写
- **Node**：处理节点，接收State返回更新后的State
- **Edge**：状态转移边，可以是条件边（根据State决定走向）
- **Checkpoint**：状态持久化，支持暂停和恢复

10. 对比CrewAI、LangGraph、AutoGen三大框架

维度	CrewAI	LangGraph	AutoGen
核心理念	角色扮演协作	状态图编排	对话驱动协作
编排方式	Crew+Flow	StateGraph	GroupChat
灵活性	中等	高	高
学习曲线	低	中	中高
生产就绪	高	高	中
适合场景	团队协作任务	复杂状态流	研究实验

B. 核心算法 (10题)

11. 什么是BKT（贝叶斯知识追踪）？

BKT是一种用概率模型估计学生知识掌握程度的方法。4个核心参数：

- $P(L_0)$：初始掌握概率（先验）
- P(T)：学习转移概率（每次练习后学会的概率）
- P(G)：猜测概率（不会但猜对）
- P(S)：失误概率（会但做错）

核心公式（答对时）： $P(L|correct) = P(L)(1-P(S)) / [P(L)(1-P(S)) + (1-P(L))P(G)]$

12. BKT和简单正确率的区别？为什么用BKT？

简单正确率 = 正确次数 / 总次数，是静态的，不考虑学习过程。

BKT的优势：

- **动态追踪**：每次练习后实时更新mastery，反映学习过程
- **考虑猜测和失误**：答对不一定会了（可能猜的），答错不一定不会（可能失误）
- **概率建模**：给出mastery的概率估计，有不确定性度量
- **学习转移**：即使答错，也考虑"练习本身就在学"

13. SM-2间隔重复算法的核心公式？

```
EF' = EF - 0.8 + 0.28 × q - 0.02 × q²  (EF范围[1.3, 2.5])
I(1) = 1天
I(2) = 6天
I(n) = I(n-1) × EF  (n>2)
```

当q < 3（回答质量差）时，重置间隔从第1天重新开始。

14. SM-2中EF（难度因子）的含义？

EF (Easiness Factor) 衡量知识点对该学生的难度：

- EF = 2.5：最容易，间隔增长最快
- EF = 1.3：最难，间隔增长最慢

- EF越大，说明学生对该知识点越熟练，复习间隔越长

15. 为什么间隔重复比集中复习效果好?

心理学中的"间隔效应"（Spacing Effect）：

- 大脑在"快要忘记"时复习，形成更强的长期记忆
- 集中复习虽然短期效果好，但遗忘速度也快
- 间隔重复利用遗忘曲线（Ebbinghaus Curve），在最佳时机触发复习
- 研究显示间隔重复的长期保留率是集中学习的3-5倍

16. 知识图谱拓扑排序的应用场景?

在本项目中，知识图谱是一个DAG（有向无环图），拓扑排序用于：

1. **确定学习顺序**：前置知识必须排在前面
2. **检查可学性**：只有前置知识都掌握了，才推荐新知识点
3. **生成补救路径**：发现薄弱点时，向上追溯所有未掌握的前置知识

算法：Kahn's BFS拓扑排序，时间复杂度O(V+E)

17. 三级提示策略的设计原理?

基于Vygotsky的最近发展区(ZPD)理论：

- Level 1 (元认知暗示)：引导学生反思自己的思考过程，如"你用到了所有已知条件吗？"
- Level 2 (脚手架引导)：给出关键步骤提示，但不给答案，如"这道题需要用到勾股定理"
- Level 3 (直接答案)：给出完整解法，仅在多次尝试失败后使用

逐级提升提示的具体程度，最大化学生自主思考的机会。

18. 苏格拉底式教学是什么？如何实现?

苏格拉底式教学：不直接传授知识，而是通过提问引导学生自己发现答案。

实现方式：

1. 根据学生mastery等级选择不同的Prompt模板
2. 初学者：用简单的类比和例子
3. 进阶者：引导发现问题关键步骤

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

4. 熟练者：提出更深层思考问题

5. 掌握者：引导归纳方法论和跨知识点联系

19. 学习状态检测的特征工程?

6种学习状态及其检测优先级（从高到低）：

状态	检测条件	响应动作
IDLE	闲置>300秒	提醒回归
FRUSTRATED	连续错误≥3次	PACE_ADJUSTMENT (slow_down)
FATIGUED	会话>45分钟 + 正确率<50%	ENCOURAGEMENT + 建议休息
BORED	正确率>90% + 连续5次正确	PACE_ADJUSTMENT (speed_up)
STRUGGLING	有错误但未达挫败阈值	提供额外支持
FOCUSED	正常学习状态	无需干预

行为特征：响应时间趋势（LinkedList保留最近20次）、错误率趋势、会话时长、闲置时间。 仅在状态变化时触发干预，鼓励消息每3次触发节流一次。

20. Beta分布在知识追踪中的作用?

Beta分布 Beta(α, β) 用于表示mastery的不确定性：

- α ≈ 成功次数 + 1
- β ≈ 失败次数 + 1
- 均值 = α/(α+β) ≈ 正确率
- 方差 ∝ 1/(α+β)：数据越多，方差越小，置信度越高

优点：是二项分布的共轭先验，贝叶斯更新非常方便。

C. 系统设计与工程 (10题)

21. 系统如何处理高并发?

Python版：asyncio事件循环 + FastAPI异步处理，单进程支持数千并发连接 Java版：教学 Agent 使用同步 `@EventListener`（主链 REST 返回前处理完毕）；需要时可另开线程池做非教学旁路 Go

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

版：goroutine（每个仅约 4KB 栈）+ channel，Publish 侧 WaitGroup 等待订阅方

22. WebSocket vs HTTP轮询?

维度	WebSocket	HTTP轮询
连接	持久连接，全双工	每次请求新建连接
延迟	毫秒级推送	取决于轮询间隔
带宽	低（无重复HTTP头）	高（每次带完整头）
服务端推送	原生支持	不支持（需长轮询）
断线处理	需要心跳+重连逻辑	天然幂等

本项目中：Python / Java / Go 均用 WebSocket 做双向实时通信；Go 另提供 SSE（`/api/v1/stream/...`），并依赖 `gorilla/websocket`。

23. 数据库设计考虑?

- 学习者模型存PostgreSQL（持久化、支持事务）
- 会话状态存Redis（快速读写、TTL过期）
- 事件历史存事件表（用于审计和回溯）
- 知识图谱可存图数据库（Neo4j）或关系表

24. 如何做Agent的可观测性?

- 每个事件有唯一ID和correlation_id，可追踪完整事件链
- 事件类型使用点分命名（如 `assessment.complete`），三端一致，便于日志检索
- 结构化日志：[Agent名称] learner=XXX, event=XXX, data=XXX
- OpenTelemetry分布式追踪（生产环境）
- 事件历史查询API（Java/Go: `GET /api/v1/events/{learnerId}`，返回 last-N 事件记录）

25. 如何测试多Agent系统?

- **单元测试**：测试每个Agent的核心逻辑（BKT更新、SM-2计算、分桶逻辑、事件类型命名）
- **集成测试**：模拟事件流，验证Agent协作链路（事件链完整触发）

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

- **端到端测试**：同一请求发送三端后端，对比事件序列和 mastery 值一致性
- **混沌测试**：随机杀死某个Agent，验证系统降级能力
- **当前覆盖**：以各语言模块测试为准（Java `src/test`；Go `go test ./...`；Python `pytest`）；勿背死具体用例数

26. 如何保证Agent的状态一致性?

本项目采用"单写者策略"：

- mastery只由Assessment Agent更新
- 复习计划只由Curriculum Agent更新
- 其他Agent只读取共享状态
- 所有状态变更通过事件通知，保证最终一致性
- 乐观锁（version字段）防止并发写冲突

27. 系统的可扩展性设计?

- **新增Agent**：只需实现BaseAgent接口+订阅事件，无需修改现有代码
- **新增知识点**：添加到知识图谱即可，Agent逻辑不变
- **水平扩展**：EventBus可替换为Kafka/RabbitMQ实现跨进程/跨机器
- **LLM替换**：Agent的LLM调用通过抽象层隔离，支持切换模型

28. 性能优化策略?

- 事件总线使用缓冲区，异步非阻塞处理
- 学习者模型缓存在内存，定期批量持久化
- 知识图谱拓扑排序结果缓存，增量更新
- Java版：EventHistoryService 使用读写锁保证并发安全，内存 last-N + `events` 表落库
- Go版：sync.RWMutex；SQLite 落用户/掌握度/事件/业务实体/critiques 等
- 三端对齐：事件链同步等待（Python asyncio.gather / Java 同步监听 / Go Publish WaitGroup）确保 REST 响应时链已处理完毕

29. 错误处理和容错设计?

- 每个Agent的事件处理用try-catch包裹，不影响其他Agent

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

- EventBus安全执行handler，捕获panic/exception
- 重试机制：LLM调用失败自动重试3次，指数退避
- 降级策略：LLM不可用时使用模板回复

30. Docker部署架构?

```
docker-compose.yml
├── python-api (FastAPI服务)
├── frontend (React + Nginx)
├── postgres (数据库)
├── redis (缓存)
└── (可选) java-api / go-api
```

D. LLM与Prompt Engineering (10题)

31. 苏格拉底式Prompt模板怎么设计的?

按学生mastery等级分级：

- beginner：最简单的语言+生活类比+最简单的例题
- developing：引导发现关键步骤+给提示而非答案
- proficient：提出深层问题+引导发现知识联系
- mastered：归纳方法论+跨知识点挑战题

每个模板都明确要求"不要直接给答案"。

32. 如何评估Prompt的效果?

- **引导率**：非直接给答案的回复占比，目标85%
- **学生后续正确率**：引导后学生再做是否能做对
- **对话轮数**：到达正确答案需要几轮引导
- **学生满意度**：（如有）问卷或评分

33. LLM幻觉(Hallucination)如何处理?

- 数学解题限定在知识图谱范围内，不做超纲内容
- 答案验证：用规则引擎校验LLM给出的数学解答

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

- Prompt中加入约束："只使用以下知识点：XXX"
- 严重幻觉时降级为模板回复

34. Prompt中的温度(Temperature)参数如何选择?

- 教学场景用低温度(0.3-0.5)：需要稳定、准确的教学内容
- 鼓励/互动用中温度(0.6-0.8)：需要一定的创造性和多样性
- 绝不用高温度(>0.9)：教育场景不能"胡说八道"

35. System Prompt和User Prompt的设计原则?

System Prompt：定义Agent角色、行为准则、约束条件（不变的部分） User Prompt：包含当前上下文、学生信息、具体问题（变化的部分）

原则：

- System Prompt要具体明确，不能太泛
- User Prompt包含必要的上下文信息
- 使用Few-shot示例展示期望的回答格式

E. 语言对比与选型 (10题)

36. Python asyncio vs Java Thread vs Go goroutine?

维度	Python asyncio	Java Thread	Go goroutine
模型	协程(协作式)	线程(抢占式)	协程(抢占式)
内存	极低	~1MB/线程	~4KB/goroutine
并发量	数万协程	数千线程	百万goroutine
调度	事件循环(单线程)	OS线程调度	Go运行时调度(M:N)
适合	I/O密集型	CPU密集型	两者皆可

37. Spring Event vs 自定义EventBus?

Spring Event：

- 优点：与 Spring IoC 深度集成，`@EventListener` 注解简洁；本项目教学链采用同步监听，保证与 REST 主链一致
- 缺点：局限于单 JVM，跨服务需要改用 MQ
- 说明：Spring 仍支持 `@Async`，但本仓库教学 Agent 已改为同步分发，勿再写成「全链路 @Async 并行」

自定义EventBus（Python版）：

- 优点：完全控制，灵活定制
- 缺点：需要自己处理线程安全、错误处理

生产环境建议：本地用Spring Event，分布式用Kafka/RabbitMQ。

38. Go channel和Java BlockingQueue的区别?

- channel：语言原生支持，select多路复用，支持方向限制（chan←, ←chan）
- BlockingQueue：标准库实现，功能完善但语法较冗长
- channel更适合CSP（通信顺序进程）模型
- BlockingQueue更适合传统生产者-消费者模型

39. 为什么Python更适合AI/ML项目?

- 最丰富的AI库生态：LangChain/LangGraph/CrewAI/AutoGen都是Python优先
- LLM SDK优先支持Python：OpenAI、Anthropic等
- 快速原型开发：动态类型+简洁语法
- 数据科学生态：numpy、pandas、scikit-learn

40. 三种语言的错误处理方式对比?

- Python：try/except + 异步异常传播
- Java：try/catch + checked exception + CompletableFuture异常
- Go：返回值错误处理(error)，panic/recover用于不可恢复错误

F. 综合与进阶 (10题)

41. 如果要支持100万用户，架构怎么改?

1. EventBus替换为Kafka（分布式消息队列）
2. Agent服务化：每个Agent独立部署为微服务
3. 学习者模型分片：按learner_id哈希分片到不同数据库
4. Redis Cluster做缓存层
5. Kubernetes编排，HPA自动扩缩容
6. CDN加速前端静态资源

42. 如果Agent之间出现死锁怎么办?

本项目的事件驱动模式天然避免死锁（异步非阻塞）。但可能出现活锁：

- 两个Agent互相触发事件，形成无限循环
- 解决：事件链长度限制 + correlation_id追踪 + 去重

43. 知识追踪还有哪些模型？BKT的局限?

- **DKT (Deep Knowledge Tracing)**：用LSTM/Transformer建模学习过程
- **IRT (Item Response Theory)**：基于题目难度和区分度的静态模型
- **SPARFA**：稀疏因子分析
- BKT局限：假设知识点独立（实际有关联），参数固定（不随时间变化）

44. 这个项目有哪些可以改进的地方?

1. 接入真实LLM，替代模板回复
2. 添加RAG（检索增强生成），用教材内容辅助教学
3. 学习者模型持久化到数据库
4. 知识图谱可视化（前端展示学习路径）
5. 用DKT替代BKT提高预测精度
6. A/B测试框架评估不同教学策略效果
7. 多语言/多学科支持

45. 如何衡量这个系统的效果?

A/B测试对比实验组（使用本系统）和对照组（传统教学）：

- 知识保留率（1周后测试成绩对比）

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

- 学习效率（达到同等水平所需时间）
- 引导率（非直接给答案的比例）
- 学生留存率（继续使用的比例）
- 学生满意度（NPS评分）

46. 事件溯源(Event Sourcing)在教育系统中的价值?

- 完整记录学生学习轨迹，可回溯任意时刻的状态
- 支持"时间旅行"Debug：回放事件重现问题
- 数据分析：挖掘学习模式和行为规律
- 审计合规：教育领域需要记录教学过程

47. 如何处理冷启动问题（新学生没有数据）?

- BKT初始mastery设为$P(L_0)=0.1$（保守估计）
- 入学测试：通过几道题快速定位水平
- 协同过滤：根据相似学生的学习轨迹推荐
- 分层默认值：按年级/学科设定不同的初始参数

48. RAG如何与多Agent结合?

Tutor Agent可以集成RAG：

1. 学生提问 → 从教材向量库检索相关内容
2. 将检索到的内容作为上下文注入Prompt
3. LLM基于教材内容生成苏格拉底式引导
4. 这样保证教学内容基于教材，减少幻觉

49. Agent的安全性考虑?

- Prompt注入防御：过滤学生输入中的恶意指令
- 内容安全：LLM输出过滤（不输出有害/不当内容）
- 数据隐私：学习者数据加密存储，GDPR合规
- API鉴权：JWT/OAuth2保护接口

50. 未来多Agent系统的发展方向?

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

- Agent自主进化：Agent能根据教学效果自动优化策略
- 跨模态Agent：支持语音、图像、视频教学
- Agent市场：像插件一样安装新Agent能力
- Agent协议标准化：A2A (Agent-to-Agent) 协议、MCP等
- 端侧Agent：在手机/Edge设备上运行轻量Agent

四、面试常见项目问题及回答

Q1: "介绍一下你这个项目"

回答要点（控制在2分钟内）：

"这是一个多Agent智能教育系统，用多 Agent（核心五类 + Critic/Debate 旁路）协作完成个性化教学。 核心问题是：传统在线教育千人一面，学生遇到困难要么直接看答案，要么放弃。 我设计了Mesh+事件驱动架构，核心 Agent 负责评估、教学、课程、提示、互动监测；旁路 Critic/Debate 做评判与辩论。 最核心的两个算法是BKT贝叶斯知识追踪和SM-2间隔重复。 教学上采用苏格拉底式引导，优先通过提问引导而非直接给答案。 我分别用Python、Java、Go三种语言实现了这个系统。"

Q2: "遇到最大的技术难点是什么？"

"最大的难点是多Agent状态一致性。多 Agent 并发处理，如何保证学习者模型的一致性？ 我参考了EACL 2026的IntelliCode论文，采用了单写者策略： 每个数据维度只有一个Agent有写权限，其他Agent通过事件总线订阅变更通知。 结合乐观锁和事件溯源，实现了既高效又一致的状态管理。"

Q3: "为什么用这个技术栈？"

"选LangGraph是因为它原生支持StateGraph状态机编排，非常适合我们的Mesh架构（本项目仅用于 Critic/Debate 旁路）。 选FastAPI因为异步支持好，原生WebSocket，API文档自动生成。 选PostgreSQL因为需要持久化学习者模型，支持JSON列存储知识状态。 Java版用Spring Boot是因为国内 Java 后端常见选型，Spring Event + 同步 `@EventListener` 实现发布-订阅并与主链等待语义对齐。 Go版用标准库是因为 goroutine+channel 天然适合 Agent 并发，并用 SQLite 落主数据与 critiques。"

Q4: "如果让你重新做，会改什么？"

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

"三个改进： 第一，接入真实LLM而不是模板回复，让教学更智能； 第二，用DKT（深度知识追踪）替代BKT，利用Transformer建模知识点间的关联； 第三，加入RAG模块，让Tutor Agent能基于教材内容精准教学，减少幻觉。"

Q5: "你在团队中的角色是什么？"

"我负责整体架构设计和核心模块开发。 具体包括：事件驱动总线的设计、BKT和SM-2算法的实现、Tutor Agent的4级苏格拉底式Prompt策略设计、Engagement Agent的6状态学习检测、知识图谱DAG+拓扑排序，以及实时通信层（Java WebSocket / Go WebSocket+SSE）。 同时我也做了技术选型决策，对比了Supervisor和Mesh两种架构模式。"

五、面试注意事项

Do（要做的）

1. **带数据说话**：可讲引导式教学、复习时机设计；勿编造未测指标
2. **讲清WHY**：为什么选这个架构？为什么用这个算法？
3. **展示深度**：提到具体论文（IntelliCode, EACL 2026）
4. **准备追问**：面试官一定会追问，准备好第二层、第三层回答
5. **画图辅助**：面试时画出架构图和事件流图

Don't（不要做的）

1. **不要只说技术名词**：不说"用了LangGraph"就完了，要说怎么用的、为什么用
2. **不要背简历**：STAR法需要自然讲述，不是朗读
3. **不要说"我负责了全部"**：具体说你做了哪个模块
4. **不要回避不会的问题**：可以说"这部分我了解的设计方向是XXX，但具体实现还在研究"
5. **不要忽视业务价值**：技术是为教育服务的，展示你理解教育场景

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

六、仓库与本地演示

面试官问「能现场画一下架构吗」之前，通常更在意你是否**真的跑过仓库**。下面与 部署指南 对齐，只保留面试前最小闭环。

代码在仓库哪里

内容	路径
Python API + 七 Agent（含 Critic/Debate）+ 总线	`code/python/`
事件与枚举	`core/event_bus.py`
编排入口	`api/orchestrator.py`、`api/main.py`
前端	`code/frontend/`
容器编排	`code/docker-compose.yml`

建议走读顺序（15～30 分钟）

1. `EventType` 枚举 → 能说出「学生提交」会触发哪类下游事件。
2. `BaseAgent` 的 `subscribed_events` / `handle_event` → 能说明「开闭原则」怎么落实在新增 Agent 上。
3. 任选一个 `*_agent.py`，对照 架构设计详解 中的事件流文字。

演示账号（密码 `demo123456`）：`student@demo.com` / `teacher@demo.com` / `admin@demo.com`。

推荐只起 Python 栈：`docker compose up -d postgres redis migrate python-api frontend`，再 `exec python-api python -m scripts.seed_demo_data`。默认 `up -d` 还会拉起 Go/Java，前端仍只反代 Python。细节见 部署指南。

本地最小命令

```bash
# 基础设施 + 迁移 + 种子
cd code && docker compose up -d postgres redis
cd python
```

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

```
cp ../.env.example .env   # 改 JWT_SECRET_KEY 等
pip install -r requirements.txt
alembic upgrade head
python -m scripts.seed_demo_data
uvicorn api.main:app --reload --port 8000

# 前端联调：VITE_USE_MOCK=false
cd ../frontend && cp .env.example .env && npm install && npm run dev
```

知识点体系

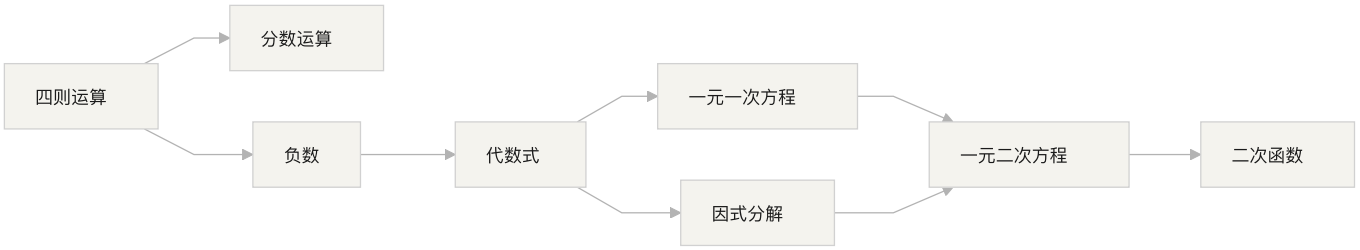
本文档描述知识图谱建模方式与当前示例数据。更大题库是扩展方向；仓库内示例约为 **20** 个节点，下文若出现「百级节点」仅为规划量级，不是已实现规模。

示例数据由 `code/python/core/knowledge_graph.py` 中 `build_sample_math_graph()` 构建，覆盖初中数学一条主链与若干分支。

知识图谱结构

知识图谱是一个 **有向无环图 (DAG)**：边表示「必须先掌握 A 再学 B」。代码里用 `KnowledgeNode.prerequisites` 表达前置，再派生出邻接表与反向邻接表。

下面用 Mermaid 画出**与示例代码一致的一小段主链**（未列出全部 20 个节点，避免图过大）：



更大范围的 ASCII 总览（便于全文检索）仍保留如下：

```
四则运算 ─┬─→ 分数运算 ──→ 概率初步
          │              └─→ 数据统计
          ├─→ 负数 ─────→ 代数式 ─┬─→ 一元一次方程 ─┬─→ 二元一次方程组
          │                       │                 ├─→ 不等式
          │                       └─→ 因式分解 ─────┴─→ 一元二次方程 ──→ 二次函
数
          │                       └─→ 数列
          │              └─→ 平面直角坐标系 ──→ 一次函数
          │
          ├─→ 勾股定理 ──→ 相似三角形
          │              └─→ 三角函数基础
          │
          └─→ 集合 ──→ 简易逻辑
```

与 `KnowledgeGraph` API 的关系

方法	作用
`add_node` / 构造函数内批量 `add_node`	注册节点与前驱边
`topological_sort()`	Kahn 算法得到全局拓扑序；若存在环会打 warning
`get_ready_nodes(mastered_ids)`	前置 ⊆ 已掌握、且自身尚未标记掌握的可学节点，**Curriculum 推荐下一知识点**时常用
`get_learning_path(target_id, mastered_ids)`	到达目标前还需补齐的最短学习序列

面试可讲：「推荐下一题」不是拍脑袋，而是 **DAG 约束下的就绪集**，与游戏技能树、课程先修同一类问题。

知识点列表

基础 (5个)

ID	名称	难度	前置知识
arithmetic	四则运算	0.1	无
fractions	分数运算	0.2	四则运算
negative_numbers	负数	0.15	四则运算
sets	集合	0.25	四则运算
logic	简易逻辑	0.3	集合

代数 (4个)

ID	名称	难度	前置知识
algebraic_expr	代数式	0.3	四则运算, 负数
factoring	因式分解	0.4	代数式
sequence	数列	0.5	代数式

方程 (4个)

ID	名称	难度	前置知识
linear_eq_1	一元一次方程	0.35	代数式
linear_eq_2	二元一次方程组	0.45	一元一次方程
quadratic_eq	一元二次方程	0.55	因式分解, 一元一次方程
inequality	不等式	0.4	一元一次方程

函数 (3个)

ID	名称	难度	前置知识
coordinate	平面直角坐标系	0.3	负数
linear_func	一次函数	0.45	一元一次方程, 坐标系
quadratic_func	二次函数	0.6	一元二次方程

几何 (2个)

ID	名称	难度	前置知识
pythagorean	勾股定理	0.35	四则运算
similar_triangle	相似三角形	0.5	勾股定理, 分数运算

三角 (1个)

官方店铺
小番薯生活集
小番薯资料铺

「小番薯生活集」官方整理，认准本店，持续更新！

ID	名称	难度	前置知识
trig_basic	三角函数基础	0.55	勾股定理, 分数运算

统计 (2个)

ID	名称	难度	前置知识
probability	概率初步	0.4	分数运算
statistics	数据统计	0.35	四则运算, 分数运算

扩展方向

同一套 DAG API 可加载更多节点（需自备题库与测试）。下列数量仅为规划量级示例，非仓库已实现规模：

- 初中数学更细粒度拆分（例如数十到近百节点）
- 高中基础专题
- 其它学科：复用 `subject` / `tags`

落地步骤建议：

1. 用表格维护「知识点 ID、名称、难度、前置 ID」，导出 JSON；
2. 加载脚本循环 `KnowledgeGraph.add_node(...)`；
3. 单测断言无环、拓扑序长度等于节点数。

更多见 架构设计详解 与 概览。